INF1103: Lab 8

Object-Oriented Relationships Part 2
(Association, Aggregation, Composition, Type Hints and Liskov Substitution)

Introduction

This lab is a little different, section 1 is all about understanding, helping you frame the 3 relationships of association, aggregation and composition a little better. Read through, understand and it will help you implement from section 2 onwards.

We also have some ***'Predict First'*** questions, to think about your code flow and outputs before you run your code, this practice can help a lot in your quiz questions! For each predict first, fill out your answers in a word document and submit with all other files for this lab.

In Lab 7, you extended your Inventory Management System using inheritance, method overriding, abstract classes and interfaces. You created PhysicalProduct and DigitalProduct as specialised types (children) of Product (parent).

This was Inheritance which focuses on the 'is-a' relationship. For example, a PhysicalProduct is-a Product. How objects work together is important, and this was also touched on last week, but this week we formalise these relationships, first by giving them names (association, aggregation and composition), and making some design decisions along the way. Remember, association and aggregation are quite similar in terms of implementation, but ownership differentiates them, whereas composition can be seen in the code and is a much stronger relationship.

In this lab, you will continue developing your Inventory Management System. You will model suppliers, inventories and customer orders, considering not just which objects refer to each other, but what the relationship means.

You will also use ***type hints*** to make your code reflect the types and relationships which should map to your UML design, apply the Liskov Substitution Principle (LSP), and use polymorphism to work with different product types through a common Product abstraction.

You will continue applying encapsulation and the SOLID principles, particularly the 'L' in SOLID and ensure you are writing good code along the way!

Learning outcomes

1. Explain and implement association, aggregation and composition.

Demonstrate polymorphism by using a common Product type to work with different product subclasses.

2. Distinguish between object references, whole-part relationships and lifecycle ownership.
3. Represent object relationships and types in a UML class diagram.
4. Use Python type hints to reflect the design represented in UML.
5. Explain and apply the Liskov Substitution Principle to an inheritance hierarchy.
6. Integrate the new classes into the existing Inventory Management System.

1. Review object relationships [Thinking / Understanding / No Coding]

Guiding question from the lecture: What happens to the part when the whole goes away? Keep asking yourself this as you work through Sections 2–4; it is the question that separates relationships of association, aggregation and composition.

Before changing your system, review the three relationships introduced in the lecture. The important distinction is not simply whether one class stores another object in an attribute. You must consider the intended relationship between the objects.

1.1 Association – objects collaborate

For the association relationship objects can know about each other or interact with each other but there is no ownership. There does not need to be a whole-part relationship, not really a strong 'has-a' relationship more 'knows-about' or 'is able to communicate/work with' relationship. An object may receive another object as a method parameter, or it may retain a reference to it.

For this lab, consider a Supplier and a Product. A supplier represents who supplied the product, but a supplier is not a part of a product, and a product is not a part of a supplier. Both can exist independently.

- A Supplier can supply many Product objects.
- A Product can exist even if its Supplier is no longer available.
- The relationship may be represented by a reference in one or both classes, depending on the design.

Association can be temporary or retained. A method parameter is one way to use an object without storing a lasting reference. For example, does a product need to know who supplies it? Does a supplier need to keep track of its products. Where possible make loose coupling of code, but where it isn't or doesn't make sense, then some awareness of other classes is ok too.

1.2 Aggregation – a whole contains independent parts

For our Aggregation relationship, we consider this a whole-part relationship (has-a relationship) in which the parts can exist independently of the whole, but the whole is made up of those parts. The whole maintains references to the parts (not optional like it is with association), but it does not control the lifecycle of the part.

In your system, an Inventory contains Product objects. Products can be added to an inventory, removed from it or transferred to another inventory. Removing a product from an inventory must not mean that the product itself is destroyed.

- An Inventory contains a collection of Product objects.
- A Product can exist before it is added to an Inventory.
- A Product can be removed or transferred without destroying it.
- An Inventory does not create or own the lifecycle of its Product objects.
    - (key differentiation between aggregation and composition)

1.3 Composition – a whole owns its parts

For a composition relationship it is also whole-part relationship (has-a relationship) but the strongest of all 3 relationships, we call it a 'terminal' relationship. The part belongs to the whole and its lifecycle is tied to that whole in the design. A part is not intended to exist independently of its owning object.

Later we will introduce Order and OrderLine classes. An Order is made up of OrderLine objects. An order line represents a particular product and quantity within that order; it is not an independent business record in this design.

- An Order creates and manages its OrderLine objects. (Order 'creates' it, so we are already seeing composition relationship)

- An OrderLine belongs to one Order.
- An OrderLine should not be transferred to another Order.
- If an Order is cancelled or removed, its OrderLine objects are also removed as part of that order.
  - This is why this is a 'terminal' relationship

Note: Python doesn't enforce relationships as strongly as other languages, but you still need to make these design decisions which should be transferred to any language based on being good software design. Your code and class responsibilities should reflect the intended relationship. Your UML diagrams should show the intended relationship.

2. Implement association: Supplier and Product

2.1 Create a Supplier class

Create a new Supplier class. Give it the following attributes:

- supplier_id: a unique identifier for the supplier.
- name: the supplier's name.
- contact_email: the supplier's contact email address.

A supplier should be able to provide information about itself. Add a display_information() method.

2.2 Associate Supplier with Product

Update Product so that it can be associated with a Supplier. For this activity, assume that each Product has one supplier. The Supplier object is created independently and then supplied to the Product when the Product is created.

Your Product constructor should accept a Supplier object, not just the supplier's name or ID. Store the reference so that the Product can access the supplier's information when required.

We will use **type hints:** A type hint shows the intended type for a variable but python doesn't enforce it, this helps articulate your intended design, but won't throw errors naturally like other languages. Example **number : float**, number is the variable, float is the type hint that isn't enforced but tells other developers about the intended type.

Use a **type hint** to express this relationship. For example, the design should communicate that the supplier attribute refers to a Supplier object.

```
class Product(ABC):
    def __init__(self, product_id: str, name: str, price: float,
                 stock_quantity: int, supplier: Supplier):
        self.supplier: Supplier = supplier
```

You may need to update the constructors of PhysicalProduct and DigitalProduct so that they pass the Supplier to Product using super().

Predict first: Before you run this test, write down what you expect to happen if you delete your supplier variable (del supplier) after creating the products. Can each Product still display its supplier information? Run it and check. If your prediction did not match, note why.

Test your association:

- Create a Supplier object.
- Create a PhysicalProduct and a DigitalProduct using that Supplier.
- Display the supplier information through each Product.

- Confirm that the Supplier and Products are separate objects and can exist independently.

Note: This is an association, not aggregation. A Product is not a whole that contains a Supplier as one of its parts. The objects are collaborating through a reference.

3. Implement aggregation: Inventory and Product

3.1 Review your Inventory class

Your Inventory class already manages Product objects. This is a useful example of aggregation: an Inventory groups Product objects, but the products can exist independently of the inventory.

Review your implementation and ensure that the Inventory stores Product objects in a collection. Use a type hint that describes the collection as containing Product objects.

```
self.products: list[Product] = []
```

The collection should be typed using the base class Product, rather than PhysicalProduct or DigitalProduct. This allows the inventory to contain either type.

3.2 Add and remove products

Ensure that your Inventory supports the following operations:

- add_product(product: Product) – add an existing Product object to the inventory.
- remove_product(product_id: str) – remove the matching Product reference from the inventory.
- find_product(product_id: str) -> Product | None – return the matching Product, or None if it cannot be found.

Use appropriate type hints for parameters and return values. The Inventory should manage the collection and locate products, but it should not implement the Product's own behaviour.

Predict first: Before step 3, write down whether you expect the Product to still work after it is removed from Inventory A, and why. Then check.

Test aggregation:

- Create a Product before creating or adding it to an Inventory.
- Add the Product to Inventory A.
- Remove it from Inventory A and confirm that the Product object can still be used.
- Add the same Product object to Inventory B.
- Confirm that removing the Product from an inventory does not remove or destroy the Product itself.

Consider what happens if an inventory is emptied. The products should not be conceptually destroyed simply because the inventory no longer contains references to them.

4. Implement composition: Order and OrderLine

4.1 Create an OrderLine class

An OrderLine represents one product and the quantity of that product requested in an order. Create a new OrderLine class with:

- product: a reference to a Product object.
- quantity: the quantity of the product ordered.

Add a method called calculate_subtotal() that returns the product price multiplied by the quantity.

```
def calculate_subtotal(self) -> float:
    return self.product.price * self.quantity
```

Use type hints to show that product is a Product object, quantity is an integer, and calculate_subtotal() returns a float.

4.2 Create an Order class

Create an Order class with the following attributes:

- order_id: a unique identifier for the order.
- order_lines: a collection of OrderLine objects.

The Order should create and manage its own OrderLine objects. Do not create an OrderLine elsewhere and pass it into the Order. Instead, provide an add_order_line() method that accepts a Product and quantity, creates the OrderLine internally, and adds it to the order.

```
class Order:
    def __init__(self, order_id: str):
        self.order_id: str = order_id
        self.order_lines: list[OrderLine] = []

    def add_order_line(self, product: Product, quantity: int) -> None:
        order_line = OrderLine(product, quantity)
        self.order_lines.append(order_line)
```

Add a calculate_total() method that calculates and returns the total value of the order by asking each OrderLine to calculate its subtotal.

The OrderLine refers to a Product, but it does not own that Product. The Product can exist independently and may also be present in an Inventory. The composition relationship is between Order and OrderLine.

4.3 Test composition

Predict first: If you add the same Product to two different Orders, how many OrderLine objects and how many Product objects will exist? Write your answer down, then check it (for example, compare objects using is or id()).

- Create an Order.
- Add two or more products to the Order using add_order_line().
- Display each order line and calculate the order total.
- Add the same Product to a different Order. Each Order should create its own OrderLine.
- Remove an order line through an Order method, or remove the Order from your test scenario. Consider why an OrderLine should not be managed independently or transferred to another Order.

Your implementation should make it clear that Order is responsible for creating and managing its OrderLine objects.

5. Type hints as part of software design

Type hints are not simply annotations added after the code has been written. In this lab, you should decide the types of attributes, parameters, collections and return values as part of your design. Those decisions should first be visible in your UML class diagram and then reflected in your Python implementation.

This is important because the design should describe the system independently of the programming language. UML is not Python-specific. A Product reference in a UML diagram could be implemented as a typed reference in Python, Java, C# or another object-oriented language.

5.1 Design the types before coding

Review the classes you have created and decide what each attribute and method should accept or return. For example:

```
class Inventory:
    def __init__(self):
        self.products: list[Product] = []

    def add_product(self, product: Product) -> None:
        self.products.append(product)

    def find_product(self, product_id: str) -> Product | None:
        for product in self.products:
            if product.product_id == product_id:
                return product
        return None
```

Think about why products is list[Product], rather than list[PhysicalProduct] or list[DigitalProduct]. The Inventory is designed to manage any Product, including new subclasses that may be introduced later.

5.2 Review all your type decisions

Review your classes and ensure that:

- Object references use the class type they refer to, e.g. supplier: Supplier.
- Collections specify the type of objects they contain, e.g. list[Product] or list[OrderLine].
- Method parameters specify the types of objects or values they expect.
- Methods specify their return types, including None where no value is returned.
- Methods that may not find an object use an appropriate optional return type, e.g. Product | None.

Do not use Any simply to avoid deciding what type an attribute or parameter should be. Choose the most appropriate type based on the design.

Important: Python type hints communicate intended types and support tools such as IDE analysis and static type checkers. They do not automatically enforce all types at runtime.

5.3 Update your UML class diagram

Before moving on, update your UML class diagram to show the types of your attributes, parameters and return values. For example:

- products: list[Product]
- supplier: Supplier
- order_lines: list[OrderLine]
- add_product(product: Product): None
- find_product(product_id: str): Product | None

Use language-independent UML type names where appropriate. For example, you may represent a collection as Product [0..*] rather than writing Python-specific list[Product]. The important point is that the diagram clearly communicates the element type and multiplicity.

6. Apply the Liskov Substitution Principle (LSP)

In Lab 7, you created PhysicalProduct and DigitalProduct as subclasses of Product. The Liskov Substitution Principle states that objects of a subclass should be usable wherever objects of the parent class are expected, without breaking the expected behaviour of the system.

In practical terms, if Inventory is designed to manage Product objects, it should not need to know whether a particular object is a PhysicalProduct or DigitalProduct in order to perform normal Product operations.

6.1 Work with the Product abstraction

Review your Inventory methods. They should accept and manage Product objects rather than checking for specific subclasses unnecessarily.

```
def display_all_products(self) -> None:
    for product in self.products:
        product.display_information()
```

The Inventory asks each Product to display its information. The appropriate implementation is selected by the object itself.

6.2 Test substitutability

Create a test function that accepts a Product and uses behaviour that every Product is expected to support.

```
def show_product_details(product: Product) -> None:
    product.display_information()
    print(product.get_fulfilment_info())
```

Call this function with both a PhysicalProduct and a DigitalProduct. Both should work without changing the function.

Also test that both product types can be added to an Inventory and used by its normal operations.

6.3 Review the expectations of the parent class

LSP is not just about whether a subclass inherits from a parent class. A subclass must preserve the expectations established by the parent class.

For example, consider the following design decisions:

- If Product provides update_stock(), both PhysicalProduct and DigitalProduct should preserve the expected meaning of updating stock.
- A subclass should not unexpectedly reject valid inputs that the parent class accepts.
- A subclass should not return a value that violates the return type or meaning promised by the parent method.
- A subclass should not change the meaning of an inherited method in a way that breaks code using Product.

Review your own Product hierarchy. Identify one expectation that Product establishes and explain how both subclasses preserve it.

Diagnose: A teammate proposes this DigitalProduct method. The program runs until Inventory calls update_stock() on a digital product.

```
class DigitalProduct(Product):
    def update_stock(self, quantity: int) -> None:
        raise NotImplementedError("Digital products have no stock")
```

Does this violate LSP? What would you change: the subclass, the parent class (Product), or the inheritance relationship? Write 2–3 sentences explaining your choice.

Do not add unnecessary subclass checks to Inventory to work around a poorly designed subclass. If a subclass cannot behave as a Product, reconsider the inheritance relationship or the responsibilities of the parent class.

7. Polymorphism: work with different Product types

Polymorphism allows objects of different classes to respond to the same method call in their own way. You have already implemented method overriding in Lab 7. In this section, you will use that behaviour through a common Product type, rather than checking which specific subclass an object belongs to.

7.1 Use a common Product interface

Your Inventory should be able to store both PhysicalProduct and DigitalProduct objects in the same collection. When you call a method such as display_information() or get_fulfilment_info() on each Product, the appropriate subclass implementation should run.

For example, your code should be able to work like this:

```
for product in inventory.products:
    product.display_information()
    print(product.get_fulfilment_info())
```

The loop does not need to ask whether each object is a PhysicalProduct or DigitalProduct. It simply calls the methods defined by the Product abstraction, and Python dispatches the call to the object's actual implementation.

7.2 Demonstrate polymorphism

Predict first: Before running your test, write down what each product will print for get_fulfilment_info(). Then compare with the actual output.

Create a test that adds at least one PhysicalProduct and one DigitalProduct to the same Inventory. Call display_information() and get_fulfilment_info() for every product through the Product collection. Check that each object produces the correct specialised behaviour.

Do not write separate loops for physical and digital products, and do not use isinstance() to decide which method to call. The purpose of this activity is to demonstrate that the common Product type provides the operations the Inventory needs, while each object supplies its own implementation.

7.3 Connect polymorphism to LSP

Polymorphism is useful when subclasses can safely stand in for their parent class. LSP helps us reason about whether that substitution is valid. If a subclass changes the expected behaviour of Product in a way that breaks the calling code, polymorphism alone will not make the design correct.

Review your show_product_details(product: Product) function from the previous section. It should work with both product types without modification. Explain how this demonstrates polymorphism and how LSP supports the design.

8. UML class diagram

Update your UML class diagram to represent the complete system. Include the classes from Lab 7 and the new classes introduced in this lab.

Your diagram should include:

- Product as an abstract class, with PhysicalProduct and DigitalProduct inheriting from it.
- Supplier associated with Product.

- Inventory aggregating Product objects.
- Order composed of OrderLine objects.
- OrderLine associated with Product.
- Attributes, parameters, return types and collection multiplicities.

Use the appropriate UML relationship notation:

- Inheritance/generalisation: solid line with a hollow triangle pointing towards the parent class.
- Association: solid line between the associated classes.
- Aggregation: solid line with a hollow diamond at the whole end.
- Composition: solid line with a filled diamond at the whole end.

Add multiplicities where appropriate, such as 1, 0..1, 1..* or 0..*. Ensure the multiplicities match the design you have implemented.

Note:

- ***Association (no ownership):*** *neither end owns the other, so ownership puts no limit on the numbers. Choose whatever the domain needs: 1, 0..1, 0..* at either end. Supplier 1 —— 0..* Product: each product has one supplier, a supplier supplies any number.*
- ***Aggregation (weak ownership):*** *there's a whole made of parts, but a part can exist outside the whole, move between wholes, or be shared. The whole end is usually **0..1** (in at most one at a time, possibly none) or **0..*** (shared). The 0 is the key: a part can exist with no whole. Inventory 0..1 ◇—— 0..* Product: a product can sit outside any inventory and can be transferred.*
- ***Composition (strong ownership):*** *the part belongs to exactly one whole and can't move or be shared, so the whole end is **exactly 1**. Order 1 ◆—— 0..* OrderLine: every line belongs to one order and never transfers.*

Remember: the diagram should communicate the design, not merely reproduce the Python code. A reader should be able to understand the relationships and expected types without needing to know Python.

AI Use: If you use AI, such as your TeamAId AI teammate, to draft or check your diagram, check every relationship it chooses against Section 1. A common mistake, by AI and people, is treating any stored reference as aggregation or composition (for example, supplier: Supplier is an association). Note one thing you accepted and one thing you changed.

9. Testing and review

Test your implementation to confirm that:

- Supplier and Product have an association and can exist independently.
- Product stores a reference to its Supplier.
- Inventory aggregates Product objects.
- Products can be removed from one Inventory and added to another without being destroyed.
- Order creates and manages its own OrderLine objects.
- OrderLine refers to Product but does not own the Product.
- The Order calculates its total using the OrderLine objects.
- Type hints are present on relevant attributes, parameters and return values.
- The types shown in the UML diagram match the types used in the Python implementation.
- Inventory can work with both PhysicalProduct and DigitalProduct through the Product abstraction.

- Both subclasses preserve the expected behaviour of Product.

Polymorphic calls through Product references invoke the correct subclass implementations.

Review your code for opportunities to keep responsibilities separated. Classes should manage their own behaviour, and relationships should reflect the intended design.

10. Reflection task (Optional)

Consider the following questions:

- ***Which of your "Predict first" answers in this lab did not match what happened? What assumption was wrong, and what did you change?***
- ***Where in your group project could you use association, aggregation or composition? Which would you choose, and why?***
- Can an object reference exist without representing aggregation or composition? Give an example from your implementation.
- Why can a Product be removed from an Inventory without being destroyed, while an OrderLine is designed to belong to an Order?
- Why should Inventory store Product references rather than separate collections for PhysicalProduct and DigitalProduct?
- How do type hints help communicate design decisions to other developers?
- Would your UML design still make sense if you implemented it in Java or C# rather than Python?
- Can a subclass technically inherit from Product but still violate LSP? Explain how.

11. Git and Docker workflow

As in the previous lab, use your existing Git and Docker workflow to manage and test your project.

- Use git status to inspect your working tree.
- Stage your project changes using git add .
- Commit your changes with a meaningful message.
- Push your changes to the remote repository.
- Run your program and tests using the established Docker workflow.
- Review your commit history using git log.

Submission requirements

Submit your updated Inventory Management System, including:

- ***Your "Predict first" notes: for each, your prediction, what happened, and what you changed if they differed (a few lines each). This can be a simple word document submitted with the other files.***
- Python source files.
- Updated UML class diagram showing association, aggregation, composition and inheritance.
- Evidence of type hints in the implementation.
- Test code or screenshots demonstrating the relationships and LSP.

Evidence that PhysicalProduct and DigitalProduct work polymorphically through Product references.

- Screenshot of your Git and Docker workflow as per previous labs.

Looking ahead

In the next lab, you will explore Error Handling to make your applications more robust when unexpected situations occur, such as invalid input, missing files or failed operations. You will also be introduced to a simple Software Design Pattern and see how a reusable design approach can help organise the responsibilities and interactions between classes.